

Credibanco Backend

Guía visual para entender qué hace este sistema,
sin necesidad de saber programar

Mayo 2026 | Tarjetas y pagos | API en puerto 8083

¿Qué es este proyecto?

Imagina un **cajero y administrador digital** que vive en un servidor. No tiene pantalla bonita para el usuario final: su trabajo es **recibir pedidos** (desde una app web, un panel de administración o una herramienta de pruebas) y **guardar o consultar información** sobre tarjetas y movimientos de dinero.



En una frase: Es el "cerebro detrás de la cortina" que permite crear tarjetas, recargar saldo, registrar compras, anular compras y consultar el historial.

¿Cómo funciona por dentro? (vista simple)

Cada pedido recorre capas, como en una empresa: quien atiende al público, quien decide las reglas, y quien archiva en el almacén.

1. **Recepción (Controllers)**
   Recibe el pedido: “crear tarjeta”, “hacer compra”, etc.
2. **Reglas de negocio (Services)**
   Valida: ¿tiene 16 dígitos? ¿hay saldo? ¿pasaron menos de 24 h para anular?
3. **Archivo (Repositories + base de datos)**
   Guarda y busca tarjetas y transacciones en PostgreSQL
4. **Respuesta**
   Devuelve el resultado al que hizo el pedido (éxito o error)

Recorrido de un pedido cualquiera



¿Qué puede hacer el sistema?

Todo se organiza en dos grandes bloques: **Tarjetas** (la "cuenta plástica digital") y **Transacciones** (cada movimiento de dinero).

💳 Tarjetas

CREAR	Registrar una tarjeta nueva con titular, tipo (crédito/débito) y saldo en cero
RECARGAR	Sumar dinero al saldo de una tarjeta existente
CONSULTAR	Ver listado de todas las tarjetas y su saldo actual

📋 Transacciones (movimientos)

RECARGA	Dejar constancia cuando se agrega saldo
COMPRA	Registrar un pago; en débito descuenta del saldo
ANULAR	Devolver una compra si fue hace menos de 24 horas
HISTORIAL	Ver todos los movimientos de una tarjeta

Ejemplo visual: un día con la tarjeta de Juan

📖 Historia de Juan (tarjeta débito)

1. Crean su tarjeta — Saldo: $0
2. → Recarga $50.000 — Saldo: $50.000
3. → Compra $10.000 — Saldo: $40.000
4. → Anula la compra — Saldo vuelve: $50.000

Cada paso queda registrado como una transacción en el historial.

Reglas importantes que aplica el sistema

- ✓ El número de tarjeta debe tener exactamente 16 dígitos
- ✓ La fecha de vencimiento debe ser al menos 3 años después de hoy
- ✓ Al crear una tarjeta, el saldo empieza en cero
- ✓ Tarjeta de débito: no puede comprar si no hay saldo suficiente
- ✓ Tarjeta de crédito: las compras no descuentan saldo (en el código actual)
- ✓ Solo se pueden anular compras exitosas, dentro de las primeras 24 horas

¿Qué información guarda?

Todo vive en una base de datos llamada **credibanco**, con dos "carpetas" principales:

🏧 TARJETA	
Número (16 dígitos)	
Nombre del titular	
Fechas creación / vencimiento	
Tipo	crédito o débito
Saldo	en dólares

Una tarjeta tiene muchas ↓

🧾 TRANSACCIÓN	
Monto	
Fecha y hora	
Tipo	recarga o compra
Estado	exitosa, anulada...
Tarjeta asociada	

⚠️ **Nota para quien mantenga el proyecto:** El archivo SQL del repositorio y el programa Java usan nombres de estados ligeramente distintos. Conviene unificarlos para evitar confusiones.

Herramientas que acompañan al proyecto

📖 **Swagger** — Página web para probar el sistema sin programar (localhost:8083/swagger-ui)

📄 **documentacion.md** — Guía técnica en el repositorio

🧪 **Pruebas automáticas** — Verifican que crear, comprar y anular funcionen

🌐 **CORS** — Permite que una app web en otro sitio se conecte al backend

Lista de mejoras recomendadas

Ordenadas por importancia. Los términos técnicos van acompañados de una explicación en lenguaje claro.

- Prioridad alta — corregir pronto
- Prioridad media — calidad y orden
- Prioridad baja — cuando el proyecto crezca

PRIORIDAD ALTA

1. Unificar la recarga en un solo camino

Hoy existen dos formas de recargar (desde "tarjetas" y desde "transacciones"). Eso confunde y puede duplicar registros. Debe quedar una sola forma oficial.

PRIORIDAD ALTA

2. Mensajes de error claros para quien consume la API

Cuando algo falla (tarjeta no existe, saldo insuficiente), la respuesta debería ser siempre un mensaje entendible con código 400/404, no un error genérico del servidor.

PRIORIDAD ALTA

3. Alinear documentación, base de datos y programa

Los mismos nombres de estados (exitosa vs realizada), formatos de fecha y rutas de la API deben coincidir en todos lados.

PRIORIDAD ALTA

4. Proteger la contraseña de la base de datos

No dejar usuario y clave visibles en archivos del proyecto. Usar variables de entorno o archivos locales que no se suban a Git.

PRIORIDAD ALTA

5. No exponer la estructura interna en las respuestas

La API debería devolver "vistas simplificadas" (DTOs), no el modelo completo de la base de datos, para mayor seguridad y claridad.

PRIORIDAD MEDIA

6. Validación automática de formularios

Rechazar datos vacíos o montos negativos antes de entrar a la lógica de negocio, con mensajes estándar.

PRIORIDAD MEDIA

7. Usar listas cerradas en lugar de texto libre

Tipo de tarjeta, tipo de movimiento y estado deberían ser opciones fijas (enum), no palabras escritas a mano que pueden tener errores de tipeo.

PRIORIDAD MEDIA

8. Registrar compras rechazadas

La documentación dice que una compra puede quedar "rechazada"; hoy solo lanza error. Conviene guardar el intento fallido en el historial.

PRIORIDAD MEDIA

9. Mejorar las pruebas

Algunos tests no prueban el servicio real o usan datos de ejemplo incorrectos. Reforzar pruebas de reglas de negocio (16 dígitos, 24 h, saldo).

PRIORIDAD MEDIA

10. README visible en la raíz del proyecto

Un archivo de bienvenida con "qué es", "cómo arrancar" y enlace a esta guía, para quien abra el repositorio por primera vez.

PRIORIDAD MEDIA

11. Migraciones versionadas de base de datos

Herramientas como Flyway permiten evolucionar el esquema sin scripts manuales sueltos.

PRIORIDAD BAJA

12. Seguridad y usuarios

Login, tokens o API keys para que no cualquiera pueda crear tarjetas o mover dinero.

PRIORIDAD BAJA

13. Evitar doble cobro en compras repetidas

Idempotencia: si el mismo pedido llega dos veces por error de red, no debe descontar dos veces.

PRIORIDAD BAJA

14. Pipeline de integración continua (CI)

Que cada cambio en Git ejecute automáticamente las pruebas.

PRIORIDAD BAJA

15. Monitoreo y logs

Para detectar fallos en producción: salud del servidor, tiempos de respuesta, errores frecuentes.

Resumen ejecutivo

Credibanco Backend es un servicio que gestiona tarjetas virtuales y sus movimientos de dinero. Recibe pedidos por internet, aplica reglas de negocio, guarda todo en PostgreSQL y responde a aplicaciones externas. Está bien estructurado para un proyecto de curso o MVP; las mejoras listadas lo acercan a un producto más profesional, seguro y fácil de mantener.